🛠️ Solution : Corriger l'état de non synchro de SQLS02

Il faut reprendre la synchronisation après avoir restauré le certificat TDE. Voici la procédure.

Étape 1 : Vérifier que le certificat TDE est bien présent sur SQLS02

```
Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    name AS CertName,
    CONVERT(VARCHAR(MAX), thumbprint, 2) AS ThumbprintHex,
    expiry_date AS ExpiryDate
FROM sys.certificates WHERE name = 'TDE_Cert'
"@
```

```
PS C:\Scripts\_Projet AlwaysOn> Get-Date

dimanche 4 octobre 2026 20:03:39



PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS02

PS C:\Scripts\_Projet AlwaysOn> Invoke-Sqlcmd -ServerInstance
"SQLS02.optimedit.eu" -TrustServerCertificate -Query @"
SELECT
    name AS CertName,
    CONVERT(VARCHAR(MAX), thumbprint, 2) AS ThumbprintHex,
    expiry_date AS ExpiryDate
FROM sys.certificates WHERE name = 'TDE_Cert'
"@

CertName ThumbprintHex                             ExpiryDate
-------- -------------                             ----------
TDE_Cert 9F5C26C1231904381471295BD1103AC06E5C0405 03/10/2027 19:58:38



PS C:\Scripts\_Projet AlwaysOn>
```

Étape 2 : Étape 1 : Suspendre les bases sur SQLS01 (Primaire)

```
#


# Sur SQLS01
# Sur SQLS01
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "SUSPEND sur $db (SQLS01)..." -ForegroundColor Cyan
    Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate `
        -Query "ALTER DATABASE [$db] SET HADR SUSPEND;"
```

```
}
```

```
SUSPEND sur DB_Direction (SQLS01)...
SUSPEND sur DB_Comptabilite (SQLS01)...
SUSPEND sur DB_Paie (SQLS01)...
SUSPEND sur DB_RH (SQLS01)...
SUSPEND sur DB_CE (SQLS01)...
SUSPEND sur DB_IT (SQLS01)...
SUSPEND sur DB_Production (SQLS01)...
SUSPEND sur DB_Formation (SQLS01)...
SUSPEND sur DB_Achat (SQLS01)...
SUSPEND sur DB_Commercial (SQLS01)...
SUSPEND sur DB_Client (SQLS01)...
SUSPEND sur DB_Juridique (SQLS01)...
SUSPEND sur DB_Blog (SQLS01)...

PS C:\Scripts\_Projet AlwaysOn>
PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS01

PS C:\Scripts\_Projet AlwaysOn>
```

Étape 2 : Reprendre les bases sur SQLS02 (Secondaire)

```
# Sur SQLS02
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "RESUME sur $db (SQLS02)..." -ForegroundColor Cyan
    try {
        Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -
TrustServerCertificate `
            -Query "ALTER DATABASE [$db] SET HADR RESUME;"
        Write-Host "  [OK] $db repris" -ForegroundColor Green
    } catch {
        Write-Host "  [ERR] Échec sur $db : $($_.Exception.Message)" -
ForegroundColor Red
    }
}
```

```
RESUME sur DB_Direction (SQLS02)...
  [OK] DB_Direction repris
RESUME sur DB_Comptabilite (SQLS02)...
  [OK] DB_Comptabilite repris
RESUME sur DB_Paie (SQLS02)...
  [OK] DB_Paie repris
RESUME sur DB_RH (SQLS02)...
  [OK] DB_RH repris
RESUME sur DB_CE (SQLS02)...
  [OK] DB_CE repris
RESUME sur DB_IT (SQLS02)...
  [OK] DB_IT repris
RESUME sur DB_Production (SQLS02)...
  [OK] DB_Production repris
RESUME sur DB_Formation (SQLS02)...
  [OK] DB_Formation repris
RESUME sur DB_Achat (SQLS02)...
  [OK] DB_Achat repris
RESUME sur DB_Commercial (SQLS02)...
  [OK] DB_Commercial repris
RESUME sur DB_Client (SQLS02)...
  [OK] DB_Client repris
RESUME sur DB_Juridique (SQLS02)...
  [OK] DB_Juridique repris
RESUME sur DB_Blog (SQLS02)...
  [OK] DB_Blog repris
```

```
PS C:\Scripts\_Projet AlwaysOn>
```

Étape 3 : Vérifier l'état après RESUME

```
Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    db.name AS DatabaseName,
    db.state_desc AS State,
    db.is_encrypted AS IsEncrypted,
    drs.synchronization_state_desc AS SyncState,
    drs.synchronization_health_desc AS SyncHealth,
    drs.is_suspended AS IsSuspended
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_databases_cluster adc ON drs.group_database_id =
adc.group_database_id
JOIN sys.databases db ON adc.database_name = db.name
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
WHERE ar.replica_server_name = 'SQLS02'
ORDER BY db.name
"@ | ft -AutoSize
```

```
DatabaseName    State  IsEncrypted SyncState         SyncHealth  IsSuspended
------------    -----  ----------- ---------         ----------  -----------
DB_Achat        ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Blog         ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_CE           ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Client       ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Commercial   ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Comptabilite ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Direction    ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Formation    ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_IT           ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Juridique    ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Paie         ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_Production   ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False
DB_RH           ONLINE       False NOT SYNCHRONIZING NOT_HEALTHY       False


PS C:\Scripts\_Projet AlwaysOn>
```

Résultat attendu :

État	Signification
`State = ONLINE` + `IsEncrypted = True` + `SyncState = SYNCHRONIZED` + `IsSuspended = False`	✅ **Problème résolu**
`State = SUSPECT` ou `SyncState = NOT SYNCHRONIZING`	❌ **Re-seeding nécessaire**

⚠️ Si RESUME ne suffit pas : Re-seeding complet

Si après `RESUME` les bases restent en `SUSPECT`, il faut **re-seeder** (recréer les bases sur SQLS02).

🔍 Diagnostic après `RESUME`

Ce que montrent les résultats

Élément	Résultat	Interprétation
`RESUME` sur les 13 bases	✅ `[OK] DB_XXX repris`	La commande a réussi
`State`	`ONLINE`	Les bases sont accessibles
`IsEncrypted`	**`False`** ❌	**TDE ne fonctionne PAS**
`SyncState`	`NOT SYNCHRONIZING` ❌	La synchronisation ne reprend pas
`SyncHealth`	`NOT_HEALTHY` ❌	L'état n'est pas sain
`IsSuspended`	`False`	Les bases ne sont plus suspendues

Diagnostic précis

Le `RESUME` a fait passer les bases de `SUSPECT` à `ONLINE`, mais :

- `IsEncrypted = False` → **Le chiffrement TDE n'est pas actif sur SQLS02.**
- `SyncState = NOT SYNCHRONIZING` → **La synchronisation ne reprend pas.**

La cause racine : Les bases sur SQLS02 ont été **chiffrées avec TDE**, mais SQLS02 **ne peut pas les déchiffrer**. Pourquoi ? Parce que la clé de chiffrement de la base (DEK) a été créée sur SQLS01 avec le certificat TDE, mais **le certificat TDE n'était pas présent sur SQLS02 au moment où les bases ont été synchronisées**.

Résultat : SQLS02 a les fichiers `.mdf`/`.ldf` chiffrés, mais **ne peut pas les déchiffrer**. Les bases sont en `ONLINE` mais **non chiffrées** (car SQL Server a désactivé TDE faute de certificat), et la synchronisation ne peut pas reprendre.

Procédure de re-seeding

1. Sur SQLS01 : Retirer les bases de l'AG

```
# Sur SQLS01
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "REMOVE DATABASE $db de l'AG..." -ForegroundColor Cyan
    Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate `
        -Query "ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] REMOVE DATABASE [$db];"
}
```

```
REMOVE DATABASE DB_Direction de l'AG...
REMOVE DATABASE DB_Comptabilite de l'AG...
REMOVE DATABASE DB_Paie de l'AG...
REMOVE DATABASE DB_RH de l'AG...
REMOVE DATABASE DB_CE de l'AG...
REMOVE DATABASE DB_IT de l'AG...
REMOVE DATABASE DB_Production de l'AG...
REMOVE DATABASE DB_Formation de l'AG...
REMOVE DATABASE DB_Achat de l'AG...
REMOVE DATABASE DB_Commercial de l'AG...
REMOVE DATABASE DB_Client de l'AG...
REMOVE DATABASE DB_Juridique de l'AG...
REMOVE DATABASE DB_Blog de l'AG...
```

2. Sur SQLS02 : Supprimer les bases corrompues

```
# Sur SQLS02
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "DROP DATABASE $db (SQLS02)..." -ForegroundColor Cyan
    try {
        Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -
TrustServerCertificate `
            -Query "IF DB_ID('$db') IS NOT NULL BEGIN ALTER DATABASE [$db] SET
SINGLE_USER WITH ROLLBACK IMMEDIATE; DROP DATABASE [$db]; END"
    } catch {
        Write-Host "  [ERR] Échec sur $db : $($_.Exception.Message)" -
ForegroundColor Red
    }
}
```

ERREURS

```
DROP DATABASE DB_Direction (SQLS02)...
Invoke-Sqlcmd : Échec de ALTER DATABASE parce qu'un verrou n'a pas pu être placé
dans la base de données 'DB_Direction'. Réessayez ultérieurement.
Échec de l'instruction ALTER DATABASE.
 Msg 5061, Niveau 16, État 1, Procédure , Ligne 1.
Au caractère Ligne:11 : 9
+         Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustSer ...
+         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

```
    + CategoryInfo          : InvalidOperation : (:) [Invoke-Sqlcmd],
SqlPowerShellSqlExecutionException
    + FullyQualifiedErrorId :
SqlError,Microsoft.SqlServer.Management.PowerShell.GetScriptCommand

DROP DATABASE DB_Comptabilite (SQLS02)...
Invoke-Sqlcmd : Échec de ALTER DATABASE parce qu'un verrou n'a pas pu être placé
dans la base de données 'DB_Comptabilite'. Réessayez ultérieurement.
Échec de l'instruction ALTER DATABASE.
Impossible de supprimer la base de données "DB_Comptabilite" car elle est en
cours d'utilisation.
 Msg 5061, Niveau 16, État 1, Procédure , Ligne 1.
Au caractère Ligne:11 : 9
+         Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustSer ...
+         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation : (:) [Invoke-Sqlcmd],
SqlPowerShellSqlExecutionException
    + FullyQualifiedErrorId :
SqlError,Microsoft.SqlServer.Management.PowerShell.GetScriptCommand

DROP DATABASE DB_Paie (SQLS02)...
```

Cause : Les bases sont **verrouillées** par l'Availability Group (elles sont encore membres de l'AG). Pour les supprimer, il faut d'abord **les retirer de l'AG** et **couper les connexions**.

Retirer les bases de l'AG sur SQLS01

Sur SQLS01, exécutez :

```powershell
# Se connecter à SQLS01
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "REMOVE DATABASE $db de l'AG..." -ForegroundColor Cyan
    try {
        Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -
TrustServerCertificate `
            -Query "ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] REMOVE DATABASE
[$db];"
        Write-Host "  [OK] $db retirée de l'AG" -ForegroundColor Green
    } catch {
        Write-Host "  [ERR] Échec sur $db : $($_.Exception.Message)" -
ForegroundColor Red
    }
}
```

```
REMOVE DATABASE DB_Direction de l'AG...
Invoke-Sqlcmd : Impossible de supprimer la base de données «DB_Direction» du
groupe de disponibilité «AG-OPTIMEDIT». La base de données n'est pas membre du
groupe de disponibilité
spécifié. Assurez-vous que le nom de la base de données et le nom du groupe de
disponibilité sont corrects, puis réessayez l'opération.
 Msg 35281, Niveau 16, État 0, Procédure , Ligne 1.
Au caractère Ligne:11 : 9
+         Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustSer ...
+         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation : (:) [Invoke-Sqlcmd],
SqlPowerShellSqlExecutionException
```

```
    + FullyQualifiedErrorId :
SqlError,Microsoft.SqlServer.Management.PowerShell.GetScriptCommand

  [OK] DB_Direction retirée de l'AG
REMOVE DATABASE DB_Comptabilite de l'AG...
Invoke-Sqlcmd : Impossible de supprimer la base de données «DB_Comptabilite» du
groupe de disponibilité «AG-OPTIMEDIT». La base de données n'est pas membre du
groupe de disponibilité
spécifié. Assurez-vous que le nom de la base de données et le nom du groupe de
disponibilité sont corrects, puis réessayez l'opération.
 Msg 35281, Niveau 16, État 0, Procédure , Ligne 1.
Au caractère Ligne:11 : 9
+         Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustSer ...
+         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation : (:) [Invoke-Sqlcmd],
SqlPowerShellSqlExecutionException
    + FullyQualifiedErrorId :
SqlError,Microsoft.SqlServer.Management.PowerShell.GetScriptCommand

  [OK] DB_Comptabilite retirée de l'AG
```

Vérifier que les bases sont retirées de l'AG

```
Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT COUNT(*) AS DbCount
FROM sys.availability_databases_cluster adc
JOIN sys.availability_groups ag ON adc.group_id = ag.group_id
WHERE ag.name = 'AG-OPTIMEDIT'
"@
WHERE ag.name = 'AG-OPTIMEDIT'
"@

DbCount
-------
      0


PS C:\Scripts\_Projet AlwaysOn>
```

Supprimer les bases corrompues sur SQLS02

Sur SQLS02, exécutez

```
# Se connecter à SQLS02
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "DROP DATABASE $db (SQLS02)..." -ForegroundColor Cyan
    try {
        $query = @"
IF DB_ID('$db') IS NOT NULL
BEGIN
    ALTER DATABASE [$db] SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE [$db];
END
"@
```

```
        Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -
TrustServerCertificate -Query $query
        Write-Host "  [OK] $db supprimée" -ForegroundColor Green
    } catch {
        Write-Host "  [ERR] Échec sur $db : $($_.Exception.Message)" -
ForegroundColor Red
    }
}
DROP DATABASE DB_Direction (SQLS02)...
  [OK] DB_Direction supprimée
DROP DATABASE DB_Comptabilite (SQLS02)...
Invoke-Sqlcmd : ALTER DATABASE n'est pas autorisé tant qu'une base de données est
dans l'état Restoring.
Échec de l'instruction ALTER DATABASE.
 Msg 5052, Niveau 16, État 1, Procédure , Ligne 3.
Au caractère Ligne:18 : 9
+         Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustSer ...
+         ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation : (:) [Invoke-Sqlcmd],
SqlPowerShellSqlExecutionException
    + FullyQualifiedErrorId :
SqlError,Microsoft.SqlServer.Management.PowerShell.GetScriptCommand

  [OK] DB_Comptabilite supprimée
DROP DATABASE DB_Paie (SQLS02)...
```

Vérifier que les bases sont supprimées sur SQLS02

```
Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT name FROM sys.databases WHERE name LIKE 'DB_%' ORDER BY name
"@
PS C:\Scripts\_Projet AlwaysOn> Invoke-Sqlcmd -ServerInstance
"SQLS02.optimedit.eu" -TrustServerCertificate -Query @"
SELECT name FROM sys.databases WHERE name LIKE 'DB_%' ORDER BY name
"@

PS C:\Scripts\_Projet AlwaysOn>
```

Retour vide

Vérifier que le certificat TDE est bien présent sur SQLS02

```
Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    name AS CertName,
    CONVERT(VARCHAR(MAX), thumbprint, 2) AS ThumbprintHex
FROM sys.certificates WHERE name = 'TDE_Cert'
"@
PS C:\Scripts\_Projet AlwaysOn> Invoke-Sqlcmd -ServerInstance
"SQLS02.optimedit.eu" -TrustServerCertificate -Query @"
SELECT
    name AS CertName,
    CONVERT(VARCHAR(MAX), thumbprint, 2) AS ThumbprintHex
FROM sys.certificates WHERE name = 'TDE_Cert'
"@

CertName ThumbprintHex
-------- -------------
TDE_Cert 9F5C26C1231904381471295BD1103AC06E5C0405
```

3. Sur SQLS01 : Re-ajouter les bases à l'AG

```powershell
# Ajouter les bases une par une
# Sur SQLS01
$databases = @(
    "DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
    "DB_IT", "DB_Production", "DB_Formation", "DB_Achat",
    "DB_Commercial", "DB_Client", "DB_Juridique", "DB_Blog"
)

foreach ($db in $databases) {
    Write-Host "Ajout de $db à l'AG..." -ForegroundColor Cyan
    try {
        Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -
TrustServerCertificate `
            -Query "ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] ADD DATABASE [$db];"
        Write-Host "  [OK] $db ajoutée à l'AG" -ForegroundColor Green
    } catch {
        Write-Host "  [ERR] Échec sur $db : $($_.Exception.Message)" -
ForegroundColor Red
    }
}
```

```
Ajout de DB_Direction à l'AG...
  [OK] DB_Direction ajoutée à l'AG
Ajout de DB_Comptabilite à l'AG...
  [OK] DB_Comptabilite ajoutée à l'AG
Ajout de DB_Paie à l'AG...
  [OK] DB_Paie ajoutée à l'AG
Ajout de DB_RH à l'AG...
  [OK] DB_RH ajoutée à l'AG
Ajout de DB_CE à l'AG...
  [OK] DB_CE ajoutée à l'AG
Ajout de DB_IT à l'AG...
  [OK] DB_IT ajoutée à l'AG
Ajout de DB_Production à l'AG...
  [OK] DB_Production ajoutée à l'AG
Ajout de DB_Formation à l'AG...
  [OK] DB_Formation ajoutée à l'AG
Ajout de DB_Achat à l'AG...
  [OK] DB_Achat ajoutée à l'AG
Ajout de DB_Commercial à l'AG...
  [OK] DB_Commercial ajoutée à l'AG
Ajout de DB_Client à l'AG...
  [OK] DB_Client ajoutée à l'AG
Ajout de DB_Juridique à l'AG...
  [OK] DB_Juridique ajoutée à l'AG
Ajout de DB_Blog à l'AG...
  [OK] DB_Blog ajoutée à l'AG
```

OU avec la syntaxe multi-bases sans crochets :

```powershell
$databases = @("DB_Direction", "DB_Comptabilite", "DB_Paie", "DB_RH", "DB_CE",
"DB_IT", "DB_Production", "DB_Formation", "DB_Achat", "DB_Commercial",
"DB_Client", "DB_Juridique", "DB_Blog")
$dbList = $databases -join ", "

Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate `
    -Query "ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] ADD DATABASE $dbList;"
```

4. Attendre la synchronisation

Le seeding automatique va copier les fichiers `.mdf`/`.ldf` de SQLS01 vers SQLS02. Pour des bases vides de 8 MB, cela prend **quelques secondes**.

Vérifiez l'état sur SQLS01:

```powershell
# Sur SQLS01 (ou depuis un poste d'administration)
Start-Sleep -Seconds 30

Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    db.name AS DatabaseName,
    db.state_desc AS State,
    db.is_encrypted AS IsEncrypted,
    drs.synchronization_state_desc AS SyncState,
    drs.synchronization_health_desc AS SyncHealth,
    drs.is_suspended AS IsSuspended
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_databases_cluster adc ON drs.group_database_id =
adc.group_database_id
JOIN sys.databases db ON adc.database_name = db.name
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
WHERE ar.replica_server_name = 'SQLS02'
ORDER BY db.name
"@ | ft -AutoSize
```

```
DatabaseName    State  IsEncrypted SyncState    SyncHealth IsSuspended
------------    -----  ----------- ---------    ---------- -----------
DB_Achat        ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Blog         ONLINE         True SYNCHRONIZED HEALTHY          False
DB_CE           ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Client       ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Commercial   ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Comptabilite ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Direction    ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Formation    ONLINE         True SYNCHRONIZED HEALTHY          False
DB_IT           ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Juridique    ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Paie         ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Production   ONLINE         True SYNCHRONIZED HEALTHY          False
DB_RH           ONLINE         True SYNCHRONIZED HEALTHY          False
```

Si `IsEncrypted = True` → ✅ **Problème résolu** (TDE fonctionne maintenant sur SQLS02).

Si `IsEncrypted = False` → ❌ **Le certificat TDE n'est toujours pas pris en compte**.

⚠️ Si `IsEncrypted = False` persiste

Si après le re-seeding, `IsEncrypted = False` sur SQLS02, cela signifie que **SQLS02 refuse toujours de déchiffrer les bases**. Causes possibles :

Cause A : Le certificat TDE n'est pas lié à la DEK

La DEK (Database Encryption Key) sur SQLS01 est chiffrée par le certificat `TDE_Cert`. Sur SQLS02, le même certificat est présent, mais SQL Server peut ne pas le reconnaître comme **le bon certificat** si les métadonnées de la DEK sont corrompues.

✅ Analyse du résultat des bases sur SQLS02

État `DatabaseName    State  IsEncrypted SyncState    SyncHealth IsSuspended`

```
------------    -----  ----------- ---------    ---------- -----------
DB_Achat        ONLINE         True SYNCHRONIZED HEALTHY          False
DB_Blog         ONLINE         True SYNCHRONIZED HEALTHY          False
DB_CE           ONLINE         True SYNCHRONIZED HEALTHY          False
..
```

Interprétation

Élément	Valeur	Signification
State	`ONLINE`	✅ Bases accessibles
IsEncrypted	`True`	✅ **TDE fonctionne**
SyncState	`SYNCHRONIZED`	✅ **Synchronisation complète**
SyncHealth	`HEALTHY`	✅ **État sain**
IsSuspended	`False`	✅ **Pas de suspension**

Les 13 bases sur SQLS02 sont maintenant :

- ✅ `ONLINE`
- ✅ `IsEncrypted = True` (TDE opérationnel)
- ✅ `SYNCHRONIZED`
- ✅ `HEALTHY`
- ✅ `IsSuspended = False`

🎯 Ce qui s'est passé

Rétrospective de l'incident :

1. **Cause initiale :** L'ordre des opérations était incorrect. Les bases ont été ajoutées à l'AG **avant** que le certificat TDE soit restauré sur SQLS02. Résultat : SQLS02 a reçu les fichiers chiffrés, mais ne pouvait pas les déchiffrer.
2. **Symptômes :** Les 13 bases étaient en `SUSPECT` puis `ONLINE` mais `IsEncrypted = False` et `NOT SYNCHRONIZING`.
3. **Diagnostic :** Le certificat TDE était bien présent sur SQLS02, mais les bases étaient dans un état incohérent.
4. **Solution appliquée :** Re-seeding complet :
   - `REMOVE DATABASE` sur SQLS01
   - `DROP DATABASE` sur SQLS02
   - `ADD DATABASE` sur SQLS01 → **seeding avec certificat en place**
5. **Résultat :** ✅ **Les bases sont maintenant chiffrées et synchronisées.**

Le re-seeding a fonctionné !

📌 Leçon importante pour le guide

Cette situation mérite une note critique dans le guide :

⚠️ **ORDRE CRITIQUE POUR TDE + ALWAYS ON**

Séquence correcte :

1. Créer les bases sur SQLS01 (6.1.0).
2. **Activer TDE sur SQLS01 (7.1.0).**
3. **Restaurer le certificat TDE sur SQLS02 (7.1.1).**
4. **Ajouter les bases à l'AG (6.2.0).** ← Le seeding se fait avec le certificat en place.

Séquence incorrecte (à éviter) :

1. Créer les bases sur SQLS01 (6.1.0).
2. Ajouter les bases à l'AG (6.2.0). ← Seeding sans certificat TDE.
3. Activer TDE sur SQLS01 (7.1.0).
4. Restaurer le certificat TDE sur SQLS02 (7.1.1). ← Trop tard, les bases sont incohérentes.

Conséquence si l'ordre est inversé : Les bases sur SQLS02 se retrouvent en `ONLINE` mais `IsEncrypted = False` et `NOT SYNCHRONIZING`. La seule solution est le **re-seeding complet** (retirer les bases de l'AG, les supprimer sur SQLS02, les re-ajouter à l'AG).

🚀 Prochaine étape : Relancer le test de failover

Maintenant que les 13 bases sont `SYNCHRONIZED` + `IsEncrypted = True` sur SQLS02, **vous pouvez relancer le test de failover** (7.1.2).

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force

.\7.1.2.Test-Failover-TDE.ps1
================================================================================
 Étape 4/4 : Rebascule vers SQLS01.optimedit.eu
================================================================================
[ERR]  Échec de la rebascule : Le réplica de disponibilité du groupe de
disponibilité « AG-OPTIMEDIT » de cette instance SQL Server ne peut pas devenir
le réplica principal. Une ou plusieurs
bases de données ne sont pas synchronisées ou ne sont pas membres du groupe de
disponibilité. Si le réplica de disponibilité utilise le mode de validation
asynchrone, pensez à exécuter un bas
culement manuel forcé (avec perte possible des données). Sinon, une fois que
toutes les bases de données secondaires locales sont membres du groupe de
disponibilité et qu'elles sont synchroni
sées, exécutez un basculement manuel programmé sur le réplica secondaire (sans
perte des données). Pour plus d'informations, consultez la documentation en ligne
SQL Server.

================================================================================
 Vérification post-exécution
================================================================================
État des réplicas après bascule :
Replica Role      OpState ConnState
------- ----      ------- ---------
SQLS01  SECONDARY         DISCONNECTED    ← PROBLÈME
SQLS02  PRIMARY   ONLINE  CONNECTED
```

Conséquence : SQLS01 est resté `DISCONNECTED`, donc :

- Il n'a pas pu recevoir les données de SQLS02.
- Il n'a pas pu se resynchroniser.
- La rebascule a échoué car SQLS01 n'était pas synchronisé.

🛠️ Actions immédiates

1. Vérifier l'état actuel de SQLS01

```
Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    ar.replica_server_name AS Replica,
    ars.role_desc AS Role,
    ars.operational_state_desc AS OpState,
    ars.connected_state_desc AS ConnState,
    ars.synchronization_health_desc AS SyncHealth
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id
JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
WHERE ag.name = 'AG-OPTIMEDIT'
ORDER BY ar.replica_server_name
"@ | ft -AutoSize
Replica Role      OpState ConnState SyncHealth
------- ----      ------- --------- ----------
SQLS01  SECONDARY ONLINE  CONNECTED HEALTHY


PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS01
```

Résultat attendu : SQLS01 doit être `SECONDARY` + `CONNECTED` (ou `DISCONNECTED` si le problème persiste).

✅ État actuel réel

Élément	Statut
SQLS01 Role	`SECONDARY` (à remettre en `PRIMARY`)
SQLS01 ConnState	`CONNECTED` ✅
SQLS01 SyncHealth	`HEALTHY` ✅
SQLS02 Role	`PRIMARY`
SQLS02 ConnState	`CONNECTED` ✅

SQLS01 est prêt à être rebasculé en `PRIMARY`.

🎯 Action immédiate : Rebasculer vers SQLS01

Étape 1 : Vérifier que les bases sont synchronisées sur SQLS01

Sur SQLS02 (qui est actuellement primaire), exécutez :

```
Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    db.name AS DatabaseName,
    drs.synchronization_state_desc AS SyncState,
    drs.synchronization_health_desc AS SyncHealth,
    drs.is_suspended AS IsSuspended
FROM sys.dm_hadr_database_replica_states drs
JOIN sys.availability_databases_cluster adc ON drs.group_database_id =
adc.group_database_id
JOIN sys.databases db ON adc.database_name = db.name
JOIN sys.availability_replicas ar ON drs.replica_id = ar.replica_id
WHERE ar.replica_server_name = 'SQLS01'
ORDER BY db.name
"@ | Format-Table DatabaseName, SyncState, SyncHealth, IsSuspended -AutoSize
```

```
DatabaseName    SyncState    SyncHealth IsSuspended
------------    ---------    ---------- -----------
DB_Achat        SYNCHRONIZED HEALTHY           False
DB_Blog         SYNCHRONIZED HEALTHY           False
DB_CE           SYNCHRONIZED HEALTHY           False
DB_Client       SYNCHRONIZED HEALTHY           False
DB_Commercial   SYNCHRONIZED HEALTHY           False
DB_Comptabilite SYNCHRONIZED HEALTHY           False
DB_Direction    SYNCHRONIZED HEALTHY           False
DB_Formation    SYNCHRONIZED HEALTHY           False
DB_IT           SYNCHRONIZED HEALTHY           False
DB_Juridique    SYNCHRONIZED HEALTHY           False
DB_Paie         SYNCHRONIZED HEALTHY           False
DB_Production   SYNCHRONIZED HEALTHY           False
DB_RH           SYNCHRONIZED HEALTHY           False
```

Résultat attendu : Les 13 bases doivent être `SYNCHRONIZED` + `HEALTHY` + `IsSuspended = False`.

Si ce n'est pas le cas, attendez 30 secondes et relancez la commande.

Étape 2 : Rebasculer vers SQLS01

Sur SQLS01, exécutez :

```
Switch-SqlAvailabilityGroup -Path
"SQLSERVER:\SQL\SQLS01\DEFAULT\AvailabilityGroups\AG-OPTIMEDIT"
```

Retour vide

Étape 3 : Vérifier que SQLS01 est bien redevenu PRIMARY

```
Start-Sleep -Seconds 10

Invoke-Sqlcmd -ServerInstance "SQLS01.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    ar.replica_server_name AS Replica,
    ars.role_desc AS Role,
    ars.connected_state_desc AS ConnState,
    ars.synchronization_health_desc AS SyncHealth
FROM sys.dm_hadr_availability_replica_states ars
JOIN sys.availability_replicas ar ON ars.replica_id = ar.replica_id
JOIN sys.availability_groups ag ON ar.group_id = ag.group_id
WHERE ag.name = 'AG-OPTIMEDIT'
ORDER BY ar.replica_server_name
"@ | Format-Table Replica, Role, ConnState, SyncHealth -AutoSize


Replica Role      ConnState SyncHealth
------- ----      --------- ----------
SQLS01  PRIMARY   CONNECTED HEALTHY
SQLS02  SECONDARY CONNECTED HEALTHY
```

📌 Leçon importante pour le script 7.1.2

Le problème rencontré est un problème de timing. Le script a tenté la rebascule **trop tôt**, avant que SQLS01 ne soit complètement reconnecté.

Améliorations à apporter au script 7.1.2 :

1. **Augmenter le timeout de rebascule** (30s → 60s).
2. **Vérifier que SQLS01 est CONNECTED** avant de tenter la rebascule.
3. **Attendre que les bases soient SYNCHRONIZED** avant de rebasculer.
4. **Réessayer la rebascule** en cas d'échec (avec un délai).

🚀 Action immédiate

Exécutez les 3 commandes ci-dessus dans l'ordre :

1. Vérifier la synchronisation sur SQLS01 (depuis SQLS02)

```powershell
# Sur SQLS02, vérifier l'état de la DEK
Invoke-Sqlcmd -ServerInstance "SQLS02.optimedit.eu" -TrustServerCertificate -
Query @"
SELECT
    db.name AS DatabaseName,
    dek.encryption_state AS EncryptionState,
    dek.key_algorithm AS KeyAlgorithm,
    dek.key_length AS KeyLength,
    c.name AS CertName,
    c.thumbprint AS CertThumbprint
FROM sys.dm_database_encryption_keys dek
JOIN sys.databases db ON dek.database_id = db.database_id
LEFT JOIN sys.certificates c ON dek.encryptor_thumbprint = c.thumbprint
ORDER BY db.name
"@ | ft -AutoSize
```

```
DatabaseName    EncryptionState KeyAlgorithm KeyLength CertName CertThumbprint
------------    --------------- ------------ --------- -------- --------------
DB_Achat                      3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Blog                       3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_CE                         3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Client                     3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Commercial                 3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Comptabilite               3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Direction                  3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Formation                  3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_IT                         3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Juridique                  3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Paie                       3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_Production                 3 AES                256 TDE_Cert {159, 92, 38, 193...}
DB_RH                         3 AES                256 TDE_Cert {159, 92, 38, 193...}
tempdb                        3 AES                256


PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS02
```